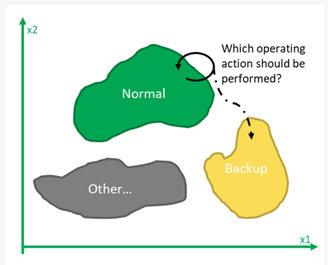
Requirement Verification with CRML and OpenModelica

Lena Buffoni[1] Adrian Pop[1] Audrey Jardin[2]

[1]Linköping University, {lena.buffoni,adrian.pop}@liu.se [2]Electricité de France, audrey.jardin@edf.fr

CRML An European Initiative for Modeling Requirements of Cyber-Physical Systems



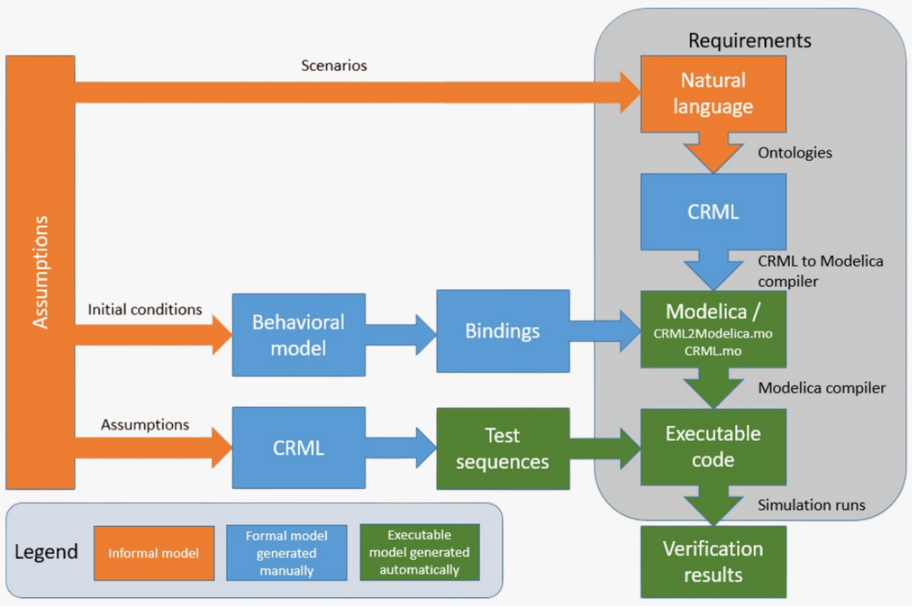
- The system should stay within the normal operating domain.
- If partial requirement 1 fails, then the system should go back to its normal operating domain within a given time delay.
- If partial requirement 2 fails, or if partial requirement 1 fails with a too high failure rate, the system should go to a safe backup state within a given time delay.
- The complete requirement made of the conjunction of partial requirements 1, 2 and 3 should be satisfied with a given probability.

A Typical Realistic Requirement for Cyber-Physical Systems

As cyber-physical systems become increasingly complex, the management and verification of requirements during design is essential. A new language called CRML (Common Requirement Modeling Language) has been created during the **European ITEA EMBrACE project** to formalize realistic dynamical requirements, but a method for developing these requirements and a framework for using them as a design aid must be defined to ease appropriation by engineers. The origin of the CRML comes the fact that the disadvantages of existing requirements modeling methods: they lack of object-orientation, have difficult mathematical syntax, often do not deal with probabilistic aspects and, most importantly, tend to use abstractions of the system in the form of state machines, which already express a kind of a behavior and is not appropriate to correctly deal with systems with strong physical aspects that evolve continuously over time. The CRML language enables alleviating these problems by allowing the formal expression of realistic dynamical requirements.

Workflow

The goal of CRML is to be a high-level language enabling the definition of requirements in quite readable but computable expressions and serving as a pivotal format capable of targeting different verification engines either by formal proof (not addressed here) or by simulation.



Workflow for Requirement Verification by Simulation

Simulation is particularly useful for requirements in the continuous-time domain: a simulation model of the tested solution provides the inputs (`=external` variables) of the CRML model to assess the value of the associated requirements over the test scenarios.

CRML Basics

A requirement in CRML is defined as an expression combining up to 4 items:

$$R = \texttt{[Where]}\ \texttt{(When)}\ \texttt{(What)}\ \texttt{[How well]}$$

- a **condition** to be checked;
- a **spatial locator** indicating on which object the condition has to be verified;
- a **time locator** defining when the condition has to be satisfied;
- (optionally) a **statistical target** to indicate with which performance the condition has to be satisfied.

A CRML requirement is therefore of a "special" Boolean built-in type, here called Boolean4 whose value at instant `t` can vary among the set `{true, false, undefined, undecided}`. The theoretical rationale could be found in **IEEE article on ETL**: a new temporal logic algebra to rigorously compute the value of requirement over continuous-time periods and to enable the combination of requirements together.

b1 and b2	true	false	undecided	undefined
true	true	false	undecided	true
false	false	false	false	false
undecided	undecided	false	undecided	undecided
undefined	true	false	undecided	undefined

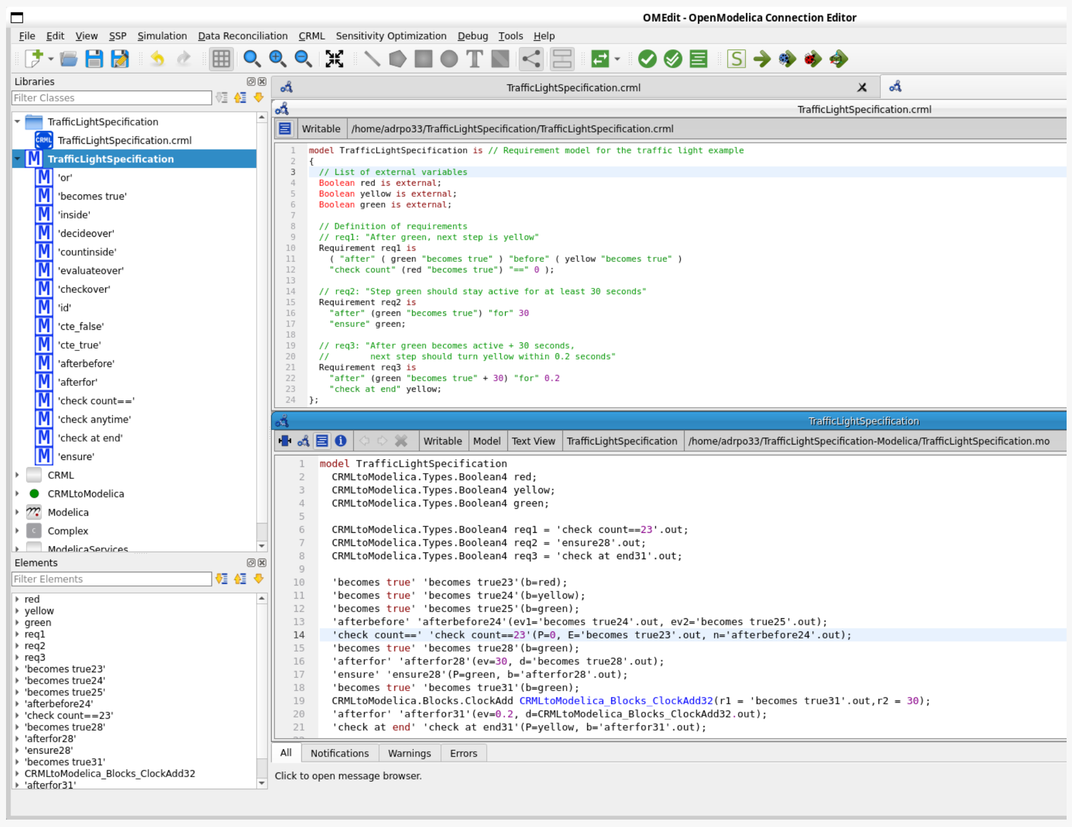
Example of Truth Table for Combining Requirements

In practice, the value of a CRML requirement is:

- `true` if the condition is satisfied over the defined time period;
- `false` if the condition is violated over the defined time period;
- `undefined` if the time locator has not been tested over the test scenario;
- `undecided` if the test scenario has finished before a decision could be made (i.e. before the condition has been violated or before the end of the time locator).

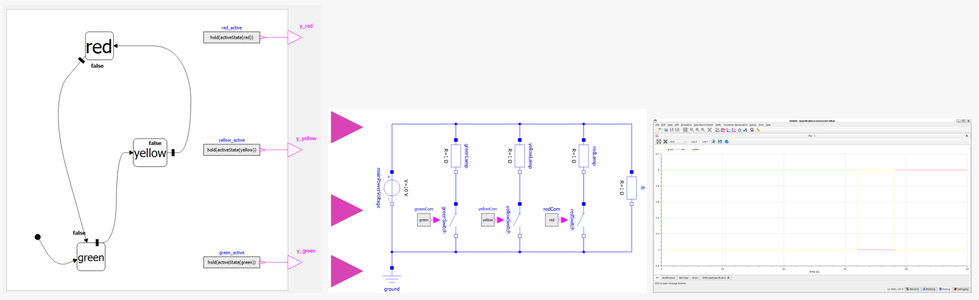
Traffic Light Example

The mission of the Traffic Light System (TLS) is to avoid accidents and fluidize traffic by minimizing congestion. The main functions are, respectively: F1) give clear instructions to each vehicle and F2) prioritize crossing durations based on road traffic. The corresponding requirements could therefore check that *"at most only one light should be on"* and *"the time spent in a light mode must be between two limits"*.



Requirement Model:CRML file display in OMEdit and its Modelica translation

Integration of the CRML tools into OpenModelica has been started within the framework of the **Open Source Modelica Consortium** and is avaliable from Openmodelica v 1.25.3. The CRML files can be loaded in the OpenModelica Connection Editor (OMEdit) and displayed with syntax highlighting. It is also possible to load entire directories containing CRML files.
The CRML files can be translated to Modelica from OMEdit via menu actions (via right click) and the translated .mo files are automatically loaded into OMEdit.



Solution Model: capture of behavior in Modelica and simulation to compute value of requirements

The generated Modelica requirement models can then be used to observe the Modelica solution model representing how the physics of the system evolve dynamically. Results can then be produced to state the satisfaction or violation of requirements over simulated time.

Tool Support

The CRML2Modelica compiler is a proof of concept compiler written in Java. It is open source and available on **OpenModelica/CRML GitHub**. Currently it supports almost all the Extended Temporal Library (ETL) library blocks which define all the operators required for computing Boolean4-status of a requirement.
The CRML library also available on GitHub implements a subset of ETL and CRML constructs as components in Modelica, so that the requirements can be directly created in Modelica.
A tutorial, examples and other documentation (language specifications, publications, ...) are also available on GitHub.

Future Work

CRML tooling is progressing to enable the formalization of realistic dynamic requirements of complex systems. Integration into OpenModelica enables a first step towards an easy access for the Modelica/FMI community to check requirements with simulation means. Visual support for requirement expression and result visualization is ongoing. Interfacing with tools for formal proofs is being investigated as well as with the **ReqIF** and **SysML v2** standards for better integration into a larger Model-Based Systems Engineering framework. Use of structural analysis techniques to statically to check requirements consistency and CRML structuration as Assume/Guarantee contracts to facilitate the generation of test scenarios from CRML constraints are also considered.

Sponsorship